

Distributed Probabilistic ECS Analogues for Daimoi-Presence Simulations

Executive summary

This report surveys real-world systems that are structurally analogous to your **distributed probabilistic Entity-Component System**: mobile particles/agents that carry high-dimensional state (your embeddings), probabilistic payloads (your Dirichlet/Beta parameters), collide/merge/transfer state, and move across **single-writer namespaces** ("Presences") with shared read access over a **Redis-like** datastore. The closest "family resemblances" appear in four mature ecosystems:

High-performance particle engines (molecular dynamics) such as LAMMPS and HOOMD-blue provide a proven blueprint for **single-writer ownership + handoff** via **spatial domain decomposition**, "ghost" replicas for neighbor interactions, and explicit migration of particles between owning partitions. ¹

Distributed agent-based simulations such as Repast HPC and Repast4Py offer a blueprint for **distributed entities + ghosting + synchronization** in MPI settings, including an explicit notion of "local agents" (owned) versus remote representations. ²

Actor and stateful-service runtimes (e.g., Microsoft Orleans, Akka, Dapr Actors) embody **single-threaded, turn-based state ownership** at the granularity of an actor/entity ("Presence-like") and use message passing + persistence patterns to avoid multi-writer races. ³

"Smart-contract-like" concurrency models (notably Solana's account locking and Sealevel runtime) demonstrate a robust general pattern: declare read/write sets upfront; execute non-conflicting transactions in parallel; serialize writes to the same state to preserve determinism. This is extremely close to "each presence owns one var-set but anyone can read." ⁴

On the storage and coordination layer, Redis Streams + consumer groups and Redis Lua/transactions provide practical building blocks for **shared read, single-writer commits, atomic handoff, at-least-once processing, and replayable logs**—but you must explicitly respect clustering constraints (same hash slot for multi-key atomicity). ⁵

The main gaps relative to your Daimoi model are: (1) **vector payload storage and update bandwidth** (embeddings are large and collision-heavy), (2) **atomic multi-key handoff** across shards/clusters, (3) **deterministic replay** under failures/at-least-once delivery, and (4) **security/isolation** of "smart-app Presence" code. These can be addressed by combining: Solana-style read/write scheduling, event-sourcing logs for replay, Redis slot-co-location design, Lua/CAS patterns, vector compression/quantization and batching, and a sandboxed execution model inspired by systems like SpacetimeDB reducers and Redis Lua sandboxing. ⁶

Real-world analogous systems in academia and industry

Below are concrete “reference exemplars.” Each includes: purpose, a mermaid architecture diagram, data model sketch, concurrency/ownership model, handoff/state-sharing mechanism, and primary-source links.

HPC particle engines with domain decomposition and particle migration

Representative systems: LAMMPS (MPI domain decomposition); HOOMD-blue (multi-GPU MPI domain decomposition). ⁷

Purpose

Simulate large particle systems by dividing space into subdomains; each compute rank owns particles in its subdomain and exchanges boundary (“ghost”) data with neighbors. ¹

Architecture (mermaid)

```
flowchart LR
    subgraph RankA [MPI Rank A (owns subdomain A)]
        AOwned[Owned particles]
        AGhost[Ghost particles from neighbors]
    end
    subgraph RankB [MPI Rank B (owns subdomain B)]
        BOwned[Owned particles]
        BGhost[Ghost particles from neighbors]
    end

    AOwned -- forward comm: boundary attrs --> BGhost
    BOwned -- forward comm: boundary attrs --> AGhost

    AOwned -- migrate when crossing boundary --> BOwned
    BOwned -- migrate when crossing boundary --> AOwned
```

Data model (sketch)

Particles are records with position/velocity plus per-particle attributes. LAMMPS explicitly distinguishes **owned atoms** and **ghost atoms** (copies used to compute forces across boundaries). ⁸

Concurrency / ownership model

Single-writer per particle: each particle is owned by exactly one rank (subdomain). Ghost copies are read-mostly replicas updated through “forward communication.” ⁸

State sharing and handoff

Handoff is explicit: when particles cross partition boundaries, they “migrate to new owning processors.” LAMMPS provides migration routines (“migrate_atoms”) that exchange per-atom data so the atom becomes owned by the new processor. ⁹

Primary sources (links)

```
https://docs.lammps.org/Developer_par_part.html
https://docs.lammps.org/Developer_par_comm.html
https://docs.lammps.org/Developer_par_neigh.html
https://docs.lammps.org/Developer_comm_ops.html
https://hoomd-blue.readthedocs.io/en/v2.9.7/mpi.html
```

Why this matches Daimoi

A daimoi is essentially an “owned particle” with rich attributes; collisions are local-neighborhood interactions; and crossing a Presence boundary is exactly “particle migration to a new owning processor,” with ghosting as the analog of shared-read visibility near boundaries. ¹⁰

MPI-distributed agent-based simulations with local ownership and ghost agents

Representative systems: Repast HPC; Repast4Py (Python). ¹¹

Purpose

Run large-scale ABMs by distributing agents across processes in an environment “in which many processes are running in parallel and memory is not shared,” with each process responsible for its local agents. ¹¹

Architecture (mermaid)

```
flowchart TB
    subgraph P0[MPI process 0]
        L0[Local agents]
        G0[Ghost agents]
    end
    subgraph P1[MPI process 1]
        L1[Local agents]
        G1[Ghost agents]
    end

    L0 -- sync/ghost update --> G1
    L1 -- sync/ghost update --> G0

    L0 -- migration/ownership transfer --> L1
    L1 -- migration/ownership transfer --> L0
```

Data model (sketch)

Agents are distributed across processes; each process holds local agents and (depending on model) maintains representations needed to compute local interactions. Repast4Py explicitly targets “MPI-distributed ABMs.” ²

Concurrency / ownership model

Single-writer per agent: each agent is “local” to one process; other processes observe via synchronized representations (ghosts). ¹²

State sharing and handoff

The model's core pattern is explicit synchronization: local state changes are propagated during sync phases. This is analogous to “Presences own vars; everyone reads” but with structured boundaries (only what is needed is shared). ¹¹

Primary sources (links)

```
https://repast.github.io/docs/repast_hpc.pdf
https://pmc.ncbi.nlm.nih.gov/articles/PMC9912342/
https://repast.github.io/repast4py.site/guide/user_guide.html
https://repast.github.io/repast_hpc.html
```

Why this matches Daimoi

Repast's clean separation of “local agents” vs “ghost agents” is a strong existing analog for: local collision handling + shared-read observation without multi-writer updates. ¹¹

Virtual actor runtimes with single-threaded state and asynchronous messaging

Representative system: Microsoft Orleans (virtual actors / grains). ¹³

Purpose

Provide a programming model for large distributed systems where each actor/grain encapsulates state and processes requests in a single-threaded manner, avoiding explicit locking in user code. The Orleans research report describes a single-thread guarantee: “at most one thread runs at a time within each activation,” so activation state is never concurrently accessed. ¹⁴

Architecture (mermaid)

```
flowchart LR
    Client --> GW[Orleans runtime / gateway]
    GW --> SiloA[Silo A]
    GW --> SiloB[Silo B]

    subgraph SiloA
        GrainX[Grain X (state + behavior)]
    end

    subgraph SiloB
        GrainY[Grain Y (state + behavior)]
    end
```

```
end
```

```
GrainX <--> GrainY
```

Data model (sketch)

A grain activation encapsulates its state and exposes methods invoked by asynchronous messages/requests. Orleans documentation describes it as actor-model-based with “virtual actor abstraction.” ¹⁵

Concurrency / ownership model

Single-writer per grain activation: by default, a grain processes each request to completion before beginning the next request, reflecting a single-threaded execution model. ¹⁶

State sharing and handoff

Orleans’ core approach is *not* shared-state reads; instead, state is accessed through message-based calls (queries/commands). However, your “read anything” requirement can be approximated via read-only grains or cached projections that are published to a shared store. The important transferable technique is strong single-writer state ownership at fine granularity. ¹⁷

Primary sources (links)

```
https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/Orleans-MSR-TR-2014-41.pdf
https://learn.microsoft.com/en-us/dotnet/orleans/overview
https://learn.microsoft.com/en-us/dotnet/orleans/grains/request-scheduling
```

Why this matches Daimoi

A Presence can be modeled as a grain: one “turn” updates its vars and owned daimoi; concurrency safety is inherited from the runtime. ¹⁸

Actor/event-sourced systems and “turn-based” concurrency

Representative systems: Akka typed actors; Dapr Actors (virtual actor building block); event-sourced entities patterns. ¹⁹

Purpose

Actors provide deferred execution and state encapsulation. Akka’s typed guide explicitly states actors “react to incoming messages sequentially, one at a time.” ²⁰

Dapr Actors provides a turn-based access model, enforcing per-actor locks so only one thread is active inside an actor at a time (unless reentrancy is enabled). ²¹

Architecture (mermaid)

```

flowchart TB
    subgraph Node1
        A1[Actor/Entity 1]
        A2[Actor/Entity 2]
    end
    subgraph Node2
        A3[Actor/Entity 3]
    end

    A1 <--> A3
    A2 --> A1

```

Data model (sketch)

Actor = (state, behavior, mailbox). Mailboxes buffer messages; processing is sequential per actor. ²²

Concurrency / ownership model

Single-threaded / turn-based per actor. Dapr describes enforcing the model via acquiring a per-actor lock at the beginning of a “turn” and releasing it at the end. ²³

State sharing and handoff

Actors typically do not expose raw shared reads. Instead, derived read models (CQRS projections) or external materializations provide shared read access. Event-sourced entities explicitly model state + events, and behavior is implemented in command/event handlers. ²⁴

Primary sources (links)

```

https://doc.akka.io/libraries/akka-core/current/typed/guide/actors-intro.html
https://docs.dapr.io/developing-applications/building-blocks/actors/actors-
overview/
https://docs.dapr.io/developing-applications/building-blocks/actors/actors-
features-concepts/
https://doc.akka.io/sdk/event-sourced-entities.html

```

Why this matches Daimoi

Your core need—**each Presence is the single writer for its var-space**—is the actor model’s “home turf.” The missing piece is “read anything,” which can be solved by externalizing immutable snapshots/projections. ¹⁹

Append-only log and event-sourcing infrastructures

Representative systems: Apache Kafka, Kafka Streams; EventStoreDB (Kurrent). ²⁵

Purpose

Provide durable, replayable logs to reconstruct state and support exactly-once or at-least-once processing depending on configuration and semantics. Kafka documentation states that consumers of a topic-partition read events “in exactly the same order as they were written.” ²⁶

EventStoreDB's documentation supports optimistic concurrency by specifying expected stream revision/version on append. ²⁷

Architecture (mermaid)

```
flowchart LR
    Producers --> Topic[Kafka topic (partitioned log)]
    Topic --> Proc[Stream processor / consumers]
    Proc --> Projections[Materialized state / projections]
    Proc --> NewEvents[New events appended]

    subgraph ES[EventStoreDB]
        Stream[Per-entity stream]
    end
    Proc <--> Stream
```

Data model (sketch)

Kafka: ordered events per partition; Kafka Streams introduces state stores and supports exactly-once processing semantics (EOS) when configured. ²⁸

EventStoreDB: per-stream event sequences; append operations optionally check expected revision/version. ²⁷

Concurrency / ownership model

Kafka ordering is per partition; multi-producer/multi-subscriber exists, but ordering is per partition and must be planned around keying. ²⁹

EventStoreDB uses optimistic concurrency via stream version checks. ²⁷

State sharing and handoff

Your system can treat collisions/handoffs as events in logs (Kafka or EventStoreDB). Then “Presence vars” become projections: fold events for a presence to reconstruct state at any point. ³⁰

Primary sources (links)

```
https://kafka.apache.org/documentation/
https://kafka.apache.org/42/design/design/
https://kafka.apache.org/37/configuration/kafka-streams-configs/
https://docs.kurrent.io/clients/node/legacy/v6.2/append-ing-events
https://www.kurrent.io/blog/exploring-the-main-features-of-eventstoredb-2/
```

Why this matches Daimoi

Your “probabilities shift on collisions” is naturally captured as an append-only, replayable stream: immutable collision events + deterministic reducer = reproducible outcomes. ³¹

Redis-backed distributed apps: Streams, consumer groups, transactions, and atomic scripts

Representative system: Redis Streams + consumer groups + Lua/transactions patterns. ³²

Purpose

Offer a shared in-memory (optionally persistent) datastore and messaging primitives. The official docs describe Streams as “append-only logs” with consumer groups, message acknowledgment, and retention operations. ³³

Architecture (mermaid)

```
flowchart TB
    subgraph Redis[Redis]
        Vars[p:{pid}:vars (hash/json)]
        Stream[bus:events (stream)]
        Inbox[p:{pid}:inbox (stream)]
        DState[d:{did}:state]
    end

    WorkerA[Presence worker A] -->|XADD| Stream
    WorkerB[Presence worker B] -->|XREADGROUP| Stream
    WorkerA -->|EVAL (atomic handoff)| DState
    WorkerA -->|HSET/JSON.SET (CAS)| Vars
```

Data model (sketch)

Streams: (id, field-value payload), tracked in consumer groups with a Pending Entries List (PEL); messages become “pending” when delivered and are removed from PEL by XACK. ³⁴

Transactions: WATCH/MULTI/EXEC provide optimistic locking “similar to CAS.” ³⁵

Lua scripts/functions: Redis provides server-side programmability; scripts run in a sandboxed context. ³⁶

Concurrency / ownership model

Redis does not inherently enforce “presence owns one namespace,” but it provides primitives (locks/leases via SET NX PX, CAS via WATCH/MULTI/EXEC, and atomic scripts) to build it. ³⁷

State sharing and handoff

Consumer groups support at-least-once delivery: messages enter the pending list and must be acknowledged. Claiming/retrying pending entries uses XCLAIM/XAUTOCLAIM. ³⁸

Critical limitation in clustered deployments: multi-key atomic operations (transactions, scripts) generally require keys to be in the same hash slot; Redis Cluster uses hash tags to colocate keys. ³⁹

Primary sources (links)

```
https://redis.io/docs/latest/develop/data-types/streams/  
https://redis.io/docs/latest/commands/xreadgroup/  
https://redis.io/docs/latest/commands/xack/  
https://redis.io/docs/latest/develop/using-commands/transactions/  
https://redis.io/docs/latest/develop/programmability/eval-intro/  
https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/  
https://redis.io/docs/latest/operate/rc/databases/configuration/clustering/
```

Why this matches Daimoi

This is the most direct route to your “Redis-like” vision: Presences maintain vars as hashes/JSON; daimoi live as keys; collisions/handoffs are stream events; and atomicity is achieved with Lua or CAS—provided you design keyhashing for single-slot multi-key updates. ⁴⁰

Smart-contract concurrency as read/write locked namespaces

Representative systems: Solana runtime / Sealevel model; Ethereum deterministic state transition. ⁴¹

Purpose

Provide deterministic replicated state transitions. Ethereum formalizes blockchain operation as a state transition function applied to transactions. ⁴²

Solana’s runtime is designed as a concurrent transaction processor: transactions declare data dependencies; transactions touching only read-only accounts can run in parallel while writes to the same account are serialized to avoid non-determinism. ⁴³

Architecture (mermaid)

```
flowchart LR  
  TxPool[Transactions] --> Scheduler[Lock scheduler]  
  Scheduler -->|parallel if disjoint| Exec[Execution cores]  
  Exec --> Accounts[(Accounts / state shards)]  
  
  Accounts --> Ledger[Deterministic ledger / replay]  
  Ledger --> Accounts
```

Data model (sketch)

State is partitioned into accounts; transactions list which accounts are read-only vs writable. ⁴⁴

Concurrency / ownership model

Multiple transactions that modify the same account must be processed serially to preserve deterministic replay; read-only access can be parallelized. ⁴⁴

State sharing and handoff

Handoff is modeled as changing which accounts/objects a transaction touches; the runtime coordinates via read/write locks to prevent conflicts. ⁴³

Primary sources (links)

```
https://docs.solana.com/validator/runtime
https://docs.solana.com/implemented-proposals/readonly-accounts
https://solana.com/news/sealevel---parallel-processing-thousands-of-smart-
contracts
https://ethereum.github.io/yellowpaper/paper.pdf
```

Why this matches Daimoi

This is arguably the cleanest real-world formalization of your requirement: a Presence var-space is like an account/namespace; collisions and deflect/diffuse are state transitions; and parallelism depends on disjoint write sets while allowing global reads. ⁴³

Smart-apps as stateful services inside a transactional host

Representative system: SpacetimeDB. ⁴⁵

Purpose

Combine database + server logic: application logic ("modules") is deployed into the database; state mutations occur only through designated reducer functions. SpacetimeDB docs state reducers are "the only way to mutate tables," and reducers run in database transactions with rollback on failure. ⁴⁶

It also supports real-time subscriptions: clients subscribe to queries and receive updates as matching rows change. ⁴⁷

Architecture (mermaid)

```
flowchart TB
    Client -->|SQL query + subscribe| Host[SpacetimeDB host]
    Client -->|call reducer| Host

    subgraph Host
        Tables[(Relational tables)]
        Reducers[Reducers (tx state transitions)]
        SubSys[Subscription engine]
    end

    Reducers --> Tables
```

```
Tables --> SubSys
SubSys --> Client
```

Data model (sketch)

Tables store durable state; reducers are the mutation interface; subscriptions stream row-level changes to clients. ⁴⁸

Concurrency / ownership model

Mutation is concentrated: “all database changes must go through reducers.” This is a disciplined single-writer-at-the-function-interface approach. ⁴⁹

Release notes emphasize reducers’ atomic/transactional/replayable characteristics and the need for isolation from observable side effects to maintain guarantees. ⁵⁰

State sharing and handoff

State is globally readable via SQL queries and subscriptions; mutation is via transactional reducers and scheduled reducers, enabling event-driven behavior. ⁵¹

Primary sources (links)

```
https://spacetimedb.com/docs/functions/reducers/
https://spacetimedb.com/docs/functions/
https://spacetimedb.com/docs/subscriptions
https://spacetimedb.com/docs/subscriptions/semantics
https://spacetimedb.com/docs/databases/
https://github.com/clockworklabs/SpacetimeDB/releases
```

Why this matches Daimoi

You described Presences as “spaces of variables... or useful algorithmic tools (Smart Apps/Contracts).” SpacetimeDB is a concrete example of *server-side state transitions* + *global read/subscription* + *strong transactional boundaries*—very close in spirit to “Presence-owned vars with read access for all.” ⁵²

Content-addressed and CRDT systems as “collision/merge” state

Representative systems: IPFS (Merkle DAG content-addressing); CRDT theory (Shapiro et al.); Automerge (local-first CRDT sync engine). ⁵³

Purpose

Enable distributed state replication and merging without centralized coordination. IPFS “merkleizes” data into a Merkle DAG for content addressing by CID. ⁵⁴

CRDTs are designed so replicas can be modified without coordinating and converge deterministically under strong eventual consistency assumptions. ⁵⁵

Automerge positions itself as a CRDT-based “version control for your data” with a sync protocol and a serialized form that can contain complete change history. ⁵⁶

Architecture (mermaid)

```
flowchart LR
  A[Replica A] <--> Sync[CRDT sync protocol]
  B[Replica B] <--> Sync

  A --> StoreA[(Content-addressed blocks)]
  B --> StoreB[(Content-addressed blocks)]
  StoreA --> CID[CID / Merkle DAG references]
  StoreB --> CID
```

Data model (sketch)

IPFS: content-addressed blocks linked as Merkle DAG nodes, identified by CIDs. ⁵⁷

CRDTs: state-based or op-based replicated data types with convergence rules. ⁵⁵

Automerger: JSON-like document structure plus change history in binary format specification. ⁵⁸

Concurrency / ownership model

Multi-writer is permitted; conflicts are resolved via mathematically defined merge rules (CRDT semantics) rather than locks. ⁵⁵

State sharing and handoff

Handoff becomes “sync/merge,” not “ownership transfer.” This is useful for parts of your system where you *want* multi-writer (e.g., global “field diffusion” aggregates) rather than strict single-writer Presence vars. ⁵⁹

Primary sources (links)

```
https://docs.ipfs.tech/concepts/lifecycle/
https://docs.ipfs.tech/concepts/merkle-dag/
https://docs.ipfs.tech/concepts/content-addressing/
https://www.csa.iisc.ac.in/~raghavan/CleanedPods2021/crdt-shapiro-2011.pdf
https://github.com/automerger/automerger
https://automerger.org/automerger-binary-format-spec/
```

Why this matches Daimoi

Your collision “mixing” is essentially a **merge operator**. CRDT design gives you rigorous tools for designing mergeable state (especially for diffusion fields) while preserving convergence. ⁵⁵

Vector / embedding indexing and storage systems

Representative systems: Faiss (library); Annoy (mmap-able read-only indexes); Weaviate, Milvus, Qdrant (vector DBs); Pinecone (managed/serverless vector DB); Redis vector search via Redis Query Engine. ⁶⁰

Purpose

Store/query embeddings, usually via Approximate Nearest Neighbor (ANN) indexes such as HNSW. ANN research such as HNSW proposes hierarchical graph-based indexing; HNSW is widely implemented in DBs including Redis vector search concepts documentation. ⁶¹

Architecture (mermaid)

```
flowchart TB
  App[Simulation services] --> VecAPI[Vector API]
  VecAPI --> Index[ANN index (e.g., HNSW/IVF)]
  VecAPI --> Meta[(Metadata store)]
  VecAPI --> Obj[(Vector/object storage)]
  Index --> Obj
  Meta --> Obj
```

Data model (sketch)

Vectors: dense float arrays plus metadata payloads and filters. Redis documentation describes indexing vector fields inside hashes or JSON objects using its query engine. ⁶²

Milvus documents IVF_PQ (quantization-based index) and HNSW (graph-based) trade-offs. ⁶³

Annoy explicitly supports large read-only mmaped indexes so many processes share memory. ⁶⁴

Concurrency / ownership model

Most vector systems are not designed for high-rate per-vector mutation (your collisions update embeddings frequently). Many are optimized for read-heavy retrieval and batched upserts, not continuous, fine-grained vector mixing. This is reflected in documentation emphasizing index build/update cost trade-offs (e.g., HNSW costly to add during build). ⁶⁵

State sharing and handoff

These systems shine as **shared read** stores: your presences can read “anything” (embeddings of other particles, presence prompt vectors, directives) quickly, but you should minimize writes/mutations to the vector DB by (a) keeping “hot” vectors local/in-redis, and (b) periodically compacting/persisting. This is an inference supported by the emphasis on read scaling and index locality in vector DB architectures. ⁶⁶

Primary sources (links)

```
https://arxiv.org/pdf/1702.08734
https://github.com/facebookresearch/faiss
https://github.com/spotify/annoy
https://docs.weaviate.io/weaviate/concepts/vector-index
https://milvus.io/docs/index.md
https://milvus.io/docs/ivf-pq.md
https://qdrant.tech/documentation/concepts/indexing/
```

```
https://redis.io/docs/latest/develop/ai/search-and-query/vectors/  
https://www.pinecone.io/blog/serverless-architecture/
```

Why this matches Daimoi

You explicitly want embeddings as collidable state. Real systems offer multiple patterns: “hot vectors near compute” (Annoy mmap; Redis vector fields) vs “shared vector DB service” (Milvus/Weaviate/Pinecone). ⁶⁷

Mapping the examples’ techniques to the Daimoi model

This mapping focuses on your specific mechanisms: **dual embeddings (seed/current)**, **Dirichlet payloads**, **2×2 similarity matrix**, **collision physics** → **probability shift**, **deflect/diffuse**, **presence-owned vars**, **atomic handoff**.

Presence-owned vars (single-writer) - Actor model / Orleans / Dapr: single-threaded per actor/grain supports single-writer state. Use a Presence-per-actor model; treat any cross-Presence writes as messages/events. ⁶⁸ - **Solana account locks:** explicit read-only vs writable state sets; serializes writes to the same account for determinism—directly analogous to “each Presence owns one set of vars; all can read.” ⁴³ - **HPC domain decomposition:** ownership of particles by subdomain already “solves” single-writer at scale, including migration handoff. ⁶⁹

Daimoi mobility and handoff - LAMMPS migration: particle moves across partition boundary → explicit migrate/exchange to new owner. This is the closest “physics-grade” analogue to your atomic handoff. ⁹ - **Repast local agents:** each process handles local agents; synchronization phases propagate needed info. Moving agent between partitions is conceptually similar to moving daimoi between Presences. ¹¹

Collision-locality and ghosting - LAMMPS neighbor lists + ghost atoms: collisions are computed locally using neighbor bins/stencils and ghost atoms; this closely matches presence-local collision processing with shared-read boundary replicas. ⁷⁰

Embeddings and 2×2 similarity matrix as state comparisons - Vector DB/ANN systems provide efficient cosine/inner-product similarity at scale, but are best used as shared-read lookups, not per-step mutation stores. ⁷¹ - **CRDT merge semantics** provide a rigorous language for “mixing” state in collisions—especially for diffusion fields or global aggregates that you do not want to strictly lock. ⁵⁵

Dirichlet parameters and probabilistic payload mixing - Existing systems rarely encode Dirichlet specifically, but many collision/mixing systems treat per-particle attributes as compositional quantities that blend on interaction (e.g., in particle simulations you can attach arbitrary per-atom attributes, and migration communicates them). This supports your notion of carrying probabilistic payload vectors. ⁷² - Event-sourcing (Kafka/EventStoreDB) is a natural substrate for making these probability updates replayable: record collision inputs (impulse + similarity matrix) and recompute deterministically. ⁷³

Deflect vs diffuse behaviors - Physics engines like Box2D and Bullet operationalize collision impulses and map forces into motion and contact effects, which you can reuse conceptually for “deflect” (impart motion/state) and “diffuse” (absorb/remove). ⁷⁴

- **SpacetimeDB reducers** plus isolation requirements mirror your need for well-defined effects: a reducer (Presence update step) must be atomic and isolated to preserve replayability. ⁷⁵

Gaps and limitations relative to your requirements

This section focuses on where the above systems *do not* natively satisfy “Daimoi + Presence with embeddings + probabilistic payload mixing + Redis-shared state” and what that implies.

Scalability vs atomic multi-key handoff in clustered Redis

Redis provides transactions and Lua scripting for atomic behavior, but in clustered environments multi-key transactions and scripts are generally constrained to keys in the same hash slot. Redis Cluster introduces hash tags specifically to colocate related keys for multi-key operations. ³⁹

For your design, a handoff that updates `d:{did}`, removes from `p:{A}:daimoi`, adds to `p:{B}:daimoi`, and enqueues to B’s inbox is a **multi-key atomic operation**—so key design must ensure co-location or accept a weaker consistency model with compensating actions. ⁷⁶

At-least-once delivery and replay determinism

Redis Streams consumer groups create a Pending Entries List; messages are pending until acknowledged by XACK, and can be claimed/reclaimed (XCLAIM/XAUTOCLAIM). This enables robustness but implies your processing should be idempotent and deterministic under re-delivery. ³⁸

Event-sourcing systems explicitly treat replay as first-class, but to exploit that you must record collision inputs precisely and ensure reducers are side-effect free (or use transactional outbox patterns). SpacetimeDB’s emphasis on reducer isolation to preserve atomic/retryable/replayable semantics is a direct warning here. ⁷⁵

Vector mutation rate and bandwidth

Vector DBs and ANN indexes are typically optimized for high-read and batched write/upsert; HNSW indexes can be expensive to update and memory-heavy. Weaviate documentation notes HNSW can be “costly when you add data during the index building process,” and Pinecone’s serverless architecture discussion emphasizes that some indices (like HNSW) must reside in memory. ⁶⁵

Your model updates embeddings and probabilities on **every collision**, which can be orders-of-magnitude higher write rate than typical embedding retrieval workloads. This strongly suggests you must separate **hot, frequently-mutated embeddings** from **cold, query-optimized memory**. ⁷⁷

“Read anything” conflicts with classic actor encapsulation

Actor systems are designed to avoid shared reads of internal state; reads are typically done via messages or projections. This mismatch is solvable (materialized views), but it is a real design divergence: naïvely allowing direct reads of every Presence’s state encourages tight coupling and “reads become writes” complexity unless you formalize snapshots/versions. ⁷⁸

Determinism and floating-point drift

HPC and physics engines often accept minor nondeterminism due to floating-point scheduling differences across hardware unless strict deterministic modes are enforced. Your embedding mixing and collision physics will likely amplify drift unless you standardize: fixed-point or quantized arithmetic for key state, deterministic ordering of collisions, and logged seeds. This is an inference: these sources establish that

determinism requires serial ordering constraints (Solana serializes writes to preserve replay determinism) and that actor/event-sourcing systems emphasize deterministic replay. ⁷⁹

Security and isolation for “smart-app Presences”

If Presences can be arbitrary “smart apps/contracts,” you need a sandbox and a capability model. Redis Lua scripts run in a sandboxed context with limited packages, and SpacetimeDB emphasizes reducers must be isolated (no observable side effects) to preserve atomic, replayable properties. ⁸⁰

Proposed architectural and algorithmic improvements for a Daimoi system

This section proposes concrete, implementable patterns, explicitly grounded in the strongest analogs: domain decomposition (LAMMPS/Repast), single-writer turns (actors), read/write locks (Solana), and atomic log + replay (Streams/Kafka/EventStoreDB). ⁸¹

Reference architecture: Presence-sharded simulation with event-sourced, lock-aware commits

High-level pattern (synthesis)

- Treat each Presence as a **single-writer shard** (actor-like). ⁸²
- Treat daimoi as **owned by exactly one Presence at a time**, like LAMMPS owned particles. ⁹
- Maintain shared-read state via immutable snapshots/projections (event-sourcing style). ⁸³
- Use Solana-style **read/write sets** to schedule collisions and “smart-app” invocations in parallel without conflicting writes. ⁴³

Edge vs local collision trade-off (your explicit design choice)

- *Local collisions*: only collide daimoi owned by same Presence. Simpler; scales well; minimizes cross-owner locking (like subdomain-local interaction). ⁸⁴
- *Edge collisions*: introduce “transit/edge Presences” analogous to boundary regions and ghost layers; collisions happen in transit. More physical fidelity; more handoffs. This mirrors domain decomposition boundary exchange and “ghost” data but lifted to higher-level semantics. ⁸

Redis schema and keyspace design for single-writer namespaces

Key principles

- Single-writer enforcement isn't automatic; implement with **leases + CAS/Lua**. ³⁷
- In Redis Cluster, design keys such that atomic Lua/transactions touch keys in the same hash slot—use hash tags carefully. ⁸⁵

Candidate schema (concrete)

- Presence-owned vars: - `p:{pid}:vars` (HASH or JSON) - `p:{pid}:ver` (string/integer version) - `p:{pid}:lease` (string with TTL) - Daimoi state: - `d:{did}:state` (HASH or JSON) - `d:{did}:ver` - `d:{did}:owner` (pid) - Streams/event log: - `bus:events` (STREAM) - `p:{pid}:inbox` (STREAM) for arrivals/tasks - Ownership index: - `p:{pid}:daimoi` (SET) of owned IDs

Slot co-location strategy (important)

If you want truly atomic multi-key handoff in Redis Cluster, you may need to colocate the “handoff keys” in one slot using a shared hash tag, but that can create hotspots and harms throughput if overused. Redis docs explicitly caution that mapping too many keys to the same hash slot can harm performance. ⁸⁶

Practical compromise: keep atomicity within a Presence's local keys (same slot by construction), and treat cross-Presence handoff as a two-phase, idempotent protocol recorded in Streams (durable log), rather than a single multi-key transaction across distant slots. This is consistent with the constraints documented for multi-key ops. ⁸⁷

Lua scripts and lease/CAS patterns

Below are patterns that align with Redis docs: SET NX PX for locks/leases; WATCH/MULTI/EXEC for optimistic locking; Lua scripts for atomic multi-step updates executed server-side. ⁸⁸

Lease acquire/renew (per Presence writer identity)

- Use `SET p:{pid}:lease <instance_id> NX PX <ttl_ms>` initially; renew with `SET ... XX PX ...` only if value matches (use Lua to check-and-renew safely). Redis documents SET NX PX as a building block for locks; distributed lock patterns are documented. ⁸⁹

Atomic handoff script (single-shard / same-slot variant)

This is the easiest conceptual form; whether it's safe in Redis Cluster depends on keyslot constraints. ⁹⁰

```
-- atomic_handoff.lua
-- KEYS:
-- 1 = d:{did}:state
-- 2 = p:{from}:daimoi
-- 3 = p:{to}:daimoi
-- 4 = p:{to}:inbox
-- ARGV:
-- 1 = did
-- 2 = from_pid
-- 3 = to_pid
-- 4 = new_owner_ver (optional)
-- 5 = event_payload_json

local owner = redis.call("HGET", KEYS[1], "owner")
if owner ~= ARGV[2] then
    return {err="not_owner"}
end

redis.call("HSET", KEYS[1], "owner", ARGV[3])
redis.call("SREM", KEYS[2], ARGV[1])
redis.call("SADD", KEYS[3], ARGV[1])
redis.call("XADD", KEYS[4], "*", "type", "daimoi_arrival", "did", ARGV[1],
"payload", ARGV[5])
```

```
return {ok="ok"}
```

Idempotent, two-phase handoff (cluster-friendly)

When keys are not in the same hash slot, you should not rely on multi-key atomicity. Redis documents that multi-key operations and transactions are restricted by slot behaviors; scripts must provide key names explicitly and are limited by cluster sharding rules. ⁹¹

Instead: 1) Writer A writes a `handoff_intent` event to `bus:events` (stream). ³³

2) Writer A marks daimoi owner as "in_transfer_to=B" (single-key update). 3) Writer B consumes event and commits ownership in its local structures, then acknowledges completion (stream ack + completion event).

4) Periodic reconciliation cleans stranded transfers.

This is the same spirit as event sourcing + at-least-once streaming: you trade atomic multi-key updates for logged intent + idempotent convergence. ⁹²

Collision processing: batching, determinism, and the 2×2 matrix

Collision locality and batching

Borrow from particle engines: neighbor-lists/bucketing to reduce $O(n^2)$ collision checks into near-linear cost within each Presence shard. LAMMPS documents binning / neighbor list construction for local neighbor lists. ⁹³

Deterministic collision ordering

To support replay: - sort collisions within a tick by (cell_id, pair_id) before applying updates - log (tick_id, pair_id, impulse J, size ratio, 2×2 similarity matrix summary, RNG seed) - apply pure update functions (no side effects), similar to the "reducer isolation" philosophy emphasized by SpacetimeDB and the deterministic replay concerns in Solana's serialization of conflicting writes. ⁹⁴

Embedding and probability storage strategy (hot vs cold)

- Hot path: store `e_seed` / `e_curr` as packed float arrays directly in the daimoi record in Redis (hash/JSON), or in worker memory with periodic checkpointing. Redis Query Engine can index vector fields inside hashes/JSON if you want shared-read similarity search. ⁹⁵

- Cold path: snapshot vectors into a vector DB (Milvus/Weaviate/Pinecone) for heavy retrieval queries, not per-collision updates; otherwise write amplification and index rebuild cost dominate. This follows the index maintenance tradeoffs described in HNSW/vector DB docs and Pinecone's discussion about in-memory index locality. ⁹⁶

Compression options - Quantization approaches like IVF_PQ are explicitly described in Milvus docs and in Faiss research/work; they are candidates for compressing "second embedding" or historical snapshots. ⁹⁷

Failure modes and consistency models: what to choose, explicitly

Recommended baseline consistency model - Single-writer per Presence vars: strong consistency within that namespace (lease + CAS). ⁹⁸

- Cross-Presence transfers: at-least-once events + idempotent apply (Streams pending list semantics). ⁹⁹

- Global diffusion fields: optionally CRDT-based if you need multi-writer without locks (Redis Active-Active uses CRDTs; CRDT theory ensures convergence). ¹⁰⁰

Key failure modes to design for - Consumer crashes after delivery but before XACK → message remains pending; must be reclaimed via XAUTOCLAIM or similar. ⁹⁹

- Presence worker loses lease mid-tick → partial updates; mitigate by versioned commits and “tick commit markers” in the event log (event sourcing style). ¹⁰¹

- Redis Cluster key mis-partitioning → multi-key operations fail with cross-slot constraints; must be planned using hash tags and slot rules. ¹⁰²

- Hot Presence (write-lock contention) → analogous to Solana “hot accounts”; mitigate by sharding vars, splitting presences, or moving some state to read-only caches. ⁴⁴

Security and isolation for smart-app Presences

Threat model: Presence code might be adversarial or buggy (infinite loops, heavy CPU, attempts to exfiltrate, attempts to perform side effects). Systems with strong replay guarantees push toward isolation:

- Redis Lua runs in a sandboxed context with limited package access. ¹⁰³
- SpacetimeDB emphasizes reducers must be isolated and cannot cause observable external side effects to preserve atomic/retryable/replayable properties. ⁵⁰

Concrete design suggestion (architecture improvement)

Run Presence “smart apps” in a sandbox (commonly WASM in modern systems), and enforce a capability model: - uphold “read anything” via read-only APIs - enforce “write only your namespace” via key-prefix capabilities - require deterministic execution: no wall-clock reads, no network I/O inside the step; side effects via queued events (transactional outbox pattern)

This is a design inference strongly supported by the reducer-isolation and sandbox themes in primary sources. ¹⁰⁴

Implementation roadmap with milestones and timeline

Prioritized milestones and effort estimates

Milestone A — Core single-writer Presence runtime (effort: medium)

Deliver: Presence worker loop with lease; versioned var commits; local daimoi ownership; basic deflect/diffuse behaviors. Grounded in Redis lease and transaction/CAS patterns. ⁹⁸

Milestone B — Event log + replay harness (effort: medium)

Deliver: append-only event stream, consumer group processing, idempotent handlers, replay-to-snapshot pipeline. Grounded in Redis Streams + XACK/PEL mechanics. ¹⁰⁵

Milestone C — Atomic/robust handoff protocol (effort: high)

Deliver: either (1) same-slot Lua atomic handoff, or (2) cluster-friendly two-phase idempotent handoff with reconciliation. Must incorporate cluster keyslot constraints. ¹⁰⁶

Milestone D — Collision engine with deterministic batching (effort: high)

Deliver: neighbor-binning per Presence; collision impulse computation; 2x2 similarity matrix calculation; Dirichlet/Beta update rules; deterministic ordering; collision event logging. Inspired by neighbor list locality in LAMMPS and determinism constraints in Solana-like serialization of conflicting writes. ¹⁰⁷

Milestone E — Vector storage tiering (effort: medium)

Deliver: hot/cold embedding plan; optional Redis vector indexing for read-mostly queries; optional external vector DB integration; compression/quantization experiments. Grounded in Redis vector docs and ANN system constraints. ¹⁰⁸

Milestone F — Smart-app sandbox and isolation (effort: high)

Deliver: Presence contracts executed in deterministic sandbox; capability-limited store access; side effects through events only. Inspired by sandboxing patterns. ¹⁰⁹

Timeline (mermaid Gantt)

```
gantt
  title Daimoi-Presence Distributed Simulation Roadmap
  dateFormat YYYY-MM-DD
  axisFormat %b %d

  section Core runtime
  Presence lease + versioned vars :a1, 2026-02-24, 21d
  Local daimoi ownership + deflect/diffuse :a2, after a1, 21d

  section Event log & replay
  Redis Streams log + consumer groups :b1, 2026-03-10, 21d
  Replay harness + snapshotting :b2, after b1, 21d

  section Handoff
  Same-slot Lua handoff (optional) :c1, 2026-03-24, 14d
  Cluster-safe 2-phase handoff + repair :c2, 2026-03-24, 35d

  section Collision engine
  Neighbor binning + deterministic ordering :d1, 2026-04-07, 28d
  2x2 similarity + Dirichlet updates :d2, after d1, 21d

  section Vector tiering
  Hot/cold embedding storage + compression :e1, 2026-04-21, 28d

  section Smart-app isolation
  Deterministic sandbox + capabilities :f1, 2026-05-05, 35d

  section Benchmarks & hardening
  Failure injection + invariants :g1, 2026-05-19, 21d
  Load benchmarks + tuning :g2, after g1, 21d
```

Prototype experiments with success criteria and suggested metrics

These are deliberately “small-scale but diagnostic,” designed to quickly validate the hardest constraints: single-writer enforcement, handoff correctness under failures, and vector mutation performance.

Prototype experiment: Redis Streams event-sourced Presence loop with at-least-once safety

Goal

Establish a robust baseline: presences process inbox events via consumer groups; update `p:{pid}:vars` with versioned commits; emit to a global `bus:events`. ¹¹⁰

Flow (mermaid)

```
flowchart TB
    In[XREADGROUP from p:{pid}:inbox] --> Handle[Apply deterministic reducer]
    Handle --> Commit[WATCH/MULTI/EXEC or Lua CAS]
    Commit --> Ack[XACK message]
    Handle --> Emit[XADD bus:events]
```

Success criteria

- No lost updates after killing workers mid-processing (recovered via pending claims). ⁹⁹
- Replaying the same event stream yields identical `p:{pid}:vars` snapshots (byte-equality if you canonicalize encoding). (Determinism constraint inferred from replay-oriented systems; grounded by event-sourcing principles and determinism concerns). ¹¹¹

Metrics

- Inbox throughput: messages/sec per presence
- Median and p99 processing latency
- Duplicate-processing rate (how often idempotency logic triggers)
- Recovery time after crash (pending backlog drained)

Prototype experiment: Atomic vs two-phase handoff under Redis Cluster constraints

Goal

- Verify handoff correctness under both: - same-slot atomic Lua handoff (ideal but slot-constrained)
- cluster-safe two-phase handoff (idempotent convergence) ⁷⁶

Flow (mermaid)

```
sequenceDiagram
    participant A as Presence A (owner)
    participant R as Redis
    participant B as Presence B (receiver)
```

```

A->>R: write handoff_intent (stream event)
A->>R: mark daimoi in_transfer_to=B
B->>R: consume intent (XREADGROUP)
B->>R: set owner=B, add to p:B:daimoi, enqueue arrival
B->>R: emit handoff_complete
A->>R: observe completion, cleanup

```

Success criteria

- No daimoi duplicated across owners after adversarial kill points
- Idempotent convergence: repeated delivery of handoff_intent does not create extra ownership entries
- Detect/repair stranded transfers within bounded time

Metrics

- Handoff latency (intent→arrival visible)
- Conflict rate/handoff retries
- Repair rate (stranded transfers per hour)

Prototype experiment: Embedding + Dirichlet update hot-path performance and storage tiering

Goal

- Measure feasibility of collision-driven embedding mutations: - store hot vectors in Redis hash/JSON fields, optionally indexed for read queries
- snapshot cold states into a vector DB (or Annoy mmap indexes for batch-built snapshots) ¹¹²

Success criteria

- Sustain target collision rate (e.g., 10k–100k collisions/sec across a small cluster) without saturating network or index rebuild
- Stable memory footprint via batching and compression/quantization for cold storage (PQ/IVF strategies are grounded in Faiss/Milvus docs). ¹¹³

Metrics

- Collisions/sec/core; collisions/sec/node
- Bytes written per collision (hot path)
- p99 read latency for “read anything” (vector similarity + metadata filters)
- Snapshot time and size (cold path)

Comparative technology tables

The tables below compare relevant candidates along the axes you requested: persistence, latency profile, vector support, transactions/atomicity, scalability, language support, and cost model. Sources are primarily vendor/official docs and foundational papers. ¹¹⁴

State storage and messaging

Technology	Persistence	Latency profile	Transactions / atomicity	Scalability	Notes for Daimoi
Redis core (hashes, sets, strings)	In-memory; persistence options exist, but core value is speed (general Redis property)	Very low typical latency; sensitive to hot keys	WATCH/MULTI/EXEC optimistic locking; Lua scripts and Functions for atomic multi-step logic ¹¹⁵	Redis Cluster horizontal; multi-key atomicity constrained by hash slots ⁹⁰	Best for hot Presence vars + hot daimoi state; requires explicit single-writer enforcement
Redis Streams	Optional trimming/retention; append-only log semantics ¹¹⁶	Low; supports blocking reads	Consumer groups with PEL, XACK; XAUTOCLAIM for recovery ⁹⁹	Scales; cluster behaviors vary and multi-stream reads require same slot for multi-key patterns ¹¹⁷	Ideal event log for collisions/handoffs; design for at-least-once + idempotency
Redis JSON + Query Engine	Structured docs; indexes on JSON/hashes ¹¹⁸	Low; indexing can be async for existing docs but sync for new/modified docs ¹¹⁹	Same Redis atomic patterns; cluster slot limits remain	Redis Stack scale depends on deployment	Convenient for storing daimoi records with vector fields + metadata filters
Apache Kafka	Durable distributed log; ordered per partition ²⁶	Higher than in-memory; optimized for throughput	Transactions/idempotence support; Kafka Streams EOS options ¹²⁰	High partition-based scalability	Strong candidate for durable replayable event log at scale (more operational overhead than Redis Streams)

Technology	Persistence	Latency profile	Transactions / atomicity	Scalability	Notes for Daimoi
NATS Core + JetStream	Core NATS best-effort at-most-once; JetStream adds persistence and stronger delivery semantics ¹²¹	Extremely low for core; JetStream adds overhead	JetStream supports “exactly once” via dedup + acks (within defined semantics) ¹²²	Scales well; subject-based	Excellent for command/event transport; if you rely on “exactly once,” study the precise semantics and time-window/dedup behavior ¹²³
EventStoreDB	Purpose-built event store; per-stream append with version checks ²⁷	Designed for event reads/writes; depends on storage	Optimistic concurrency via expected revision/version ²⁷	Scales by stream modeling + projections	Good if you want strict event-sourcing discipline for Presences and daimoi

Vector and embedding storage

Technology	Vector support	Mutation friendliness	Typical strengths	Notable limitations for Daimoi
Redis vector search (Query Engine)	Built-in vector fields + filtering over metadata ¹²⁴	Moderate: good for CRUD, but collision-heavy writes can still be expensive	Unified hot store + vector retrieval	Must manage update rate; indexing cost under heavy mutation is workload-dependent
Faiss	ANN algorithms + GPU support; research-grade performance ¹²⁵	Good for batch builds + query; not a DB	Fast similarity search; many indexing options	Lacks DB features (transactions, multi-tenant, persistence orchestration)
Annoy	mmapped read-only index shared across processes ⁶⁴	Low (read-only index after build)	Cheap shared read snapshots	Not suitable for per-collision updates; best as snapshot artifact
Milvus	Multiple ANNS index families incl. IVF_PQ and HNSW ⁶³	Moderate: supports CRUD/upserts, but index tradeoffs apply	Open-source scalable vector DB	Operational complexity; memory overhead (HNSW) noted in docs ¹²⁶

Technology	Vector support	Mutation friendliness	Typical strengths	Notable limitations for Daimoi
Weaviate	Vector DB with HNSW/flat/dynamic index options ¹²⁷	Moderate	Good developer ergonomics; indexing options	HNSW build/update cost tradeoffs explicitly called out ¹²⁸
Qdrant	Dense vector index uses HNSW; payload filtering indexes ¹²⁹	Moderate	Strong filtering model	HNSW-only dense index (per docs) may constrain choices ¹³⁰
Pinecone	Managed/serverless vector DB; architecture discusses shard/index locality and object storage engine ¹³¹	Moderate; favors batch-like patterns	Fully managed, high reliability	More expensive/black-box; still subject to index locality and in-memory constraints ¹³²

Actor/stateful-service runtimes

Framework	Concurrency model	Persistence story	Fit for Presence-owned vars
Microsoft Orleans	Single-threaded per grain activation by default ¹⁴	Pluggable storage; patterns for distributed state	Strong: Presences map naturally to grains; shared-read requires projections
Akka	Actors process messages sequentially, one at a time ¹³³	Akka Persistence for event-sourced actors ¹³⁴	Strong: matches single-writer; integrates event-sourcing style
Dapr Actors	Turn-based per-actor lock; single-threaded access per actor instance ²³	Backed by configured state store	Good for Presence service boundaries; “read anything” via shared store
Actix (Rust)	Actor framework; message exchange; actors can be scheduled across threads via arbiters ¹³⁵	Application-defined	Useful for implementing Presence workers in Rust
Cloudstate (archived)	Explored event sourcing + CRDT + stateful serverless; project no longer active ¹³⁶	Conceptual influence remains	Useful as design inspiration; not a current production choice

Simulation and physics engines

System	Core model	Parallelism model	Notes for Daimoi
LAMMPS	Particle simulation	MPI spatial decomposition; owned + ghost atoms; particle migration ¹³⁷	Best template for ownership/handoff/collision-locality
HOOMD-blue	Particle simulation (GPU)	MPI domain decomposition; each GPU assigned a subdomain ¹³⁸	Strong template for GPU-first daimoi physics
Repast HPC / Repast4Py	Agent-based modeling	Distributed agents across processes; local responsibility; MPI ¹¹	Best template for ghosted ABM semantics
NetLogo	ABM environment	Primarily single-process educational/research environment ¹³⁹	Useful for prototyping rules; not a distributed core
MASON	ABM library	Single-process discrete-event; emphasizes reproducibility ¹⁴⁰	Good for deterministic rule prototyping; distribution is external
Box2D / Bullet	Collision + rigid body dynamics	Local engine; distribution external ¹⁴¹	Useful for collision impulse semantics for deflect/diffuse mapping

Embedded Lua snippets and flowcharts for handoff and collision processing

Collision processing pipeline (mermaid)

```

flowchart TB
    Start[Tick start] --> Load[Load owned daimoi set]
    Load --> Bin[Spatial binning / neighbor list]
    Bin --> Pairs[Enumerate collision pairs]
    Pairs --> Sort[Deterministic sort pairs]
    Sort --> ForEach[For each pair]
    ForEach --> Matrix[Compute 2x2 similarity matrix]
    Matrix --> Physics[Compute impulse J, size ratio]
    Physics --> Update[Update embeddings + Dirichlet/Beta params]
    Update --> Emit[Append collision event to log]
    Emit --> Commit[Write updated states (CAS)]
    Commit --> Done[Tick end]

```

This pipeline is a synthesis of: neighbor-list locality (LAMMPS), deterministic serialization of conflicting writes (Solana), and replayable, side-effect-free reducer steps (event sourcing / SpacetimeDB-style constraints). ¹⁴²

Example: presence var CAS update using WATCH/MULTI/EXEC (pseudo-Lua alternative)

Redis documents WATCH/MULTI/EXEC as optimistic locking similar to CAS. ³⁵

In practice you would: - WATCH `p:{pid}:ver` - read vars + ver - compute next vars + ver+1 - MULTI: write vars + new ver - EXEC: if fails, retry

(When clustered, keep the transaction within one hash slot.) ⁹⁰

Entities

Microsoft ¹⁴³ Apache Software Foundation ¹⁴⁴ Google ¹⁴⁵ Northwestern University ¹⁴⁶ George Mason University ¹⁴⁷ Spotify ¹⁴⁸ Pinecone ¹⁴⁹ Zilliz ¹⁵⁰ Clockwork Labs ¹⁵¹ Lightbend ¹⁵² Solana Labs ¹⁵³

¹ ⁷ ⁶⁹ ¹³⁷ 4.4.1. Partitioning

https://docs.lammps.org/Developer_par_part.html?utm_source=chatgpt.com

² ¹¹ ¹² https://repast.github.io/docs/repast_hpc.pdf

https://repast.github.io/docs/repast_hpc.pdf

³ ¹³ ¹⁴ ⁶⁸ ¹⁵² Distributed Virtual Actors for Programmability and Scalability

https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/Orleans-MSR-TR-2014-41.pdf?utm_source=chatgpt.com

⁴ ⁶ ⁴¹ ⁴³ ¹⁴³ ¹⁴⁵ <https://docs.solana.com/validator/runtime>

<https://docs.solana.com/validator/runtime>

⁵ ³² ³³ ⁴⁰ ¹⁰⁵ ¹¹⁰ ¹¹⁴ ¹¹⁶ ¹⁵⁰ <https://redis.io/docs/latest/develop/data-types/streams/>

<https://redis.io/docs/latest/develop/data-types/streams/>

⁸ ⁹ ¹⁰ https://docs.lammps.org/Developer_par_comm.html

https://docs.lammps.org/Developer_par_comm.html

¹⁵ ¹⁷ Orleans overview - .NET

https://learn.microsoft.com/en-us/dotnet/orleans/overview?utm_source=chatgpt.com

¹⁶ ¹⁸ ⁸² ¹⁵³ <https://learn.microsoft.com/en-us/dotnet/orleans/grains/request-scheduling>

<https://learn.microsoft.com/en-us/dotnet/orleans/grains/request-scheduling>

¹⁹ ²⁰ ²² ⁷⁸ ¹³³ ¹⁴⁶ <https://doc.akka.io/libraries/akka-core/current/typed/guide/actors-intro.html>

<https://doc.akka.io/libraries/akka-core/current/typed/guide/actors-intro.html>

²¹ <https://v1-7.docs.dapr.io/developing-applications/building-blocks/actors/actors-overview/>

<https://v1-7.docs.dapr.io/developing-applications/building-blocks/actors/actors-overview/>

²³ <https://docs.dapr.io/developing-applications/building-blocks/actors/actors-features-concepts/>

<https://docs.dapr.io/developing-applications/building-blocks/actors/actors-features-concepts/>

²⁴ <https://doc.akka.io/sdk/event-sourced-entities.html>

<https://doc.akka.io/sdk/event-sourced-entities.html>

25 26 29 73 83 147 <https://kafka.apache.org/documentation/>

<https://kafka.apache.org/documentation/>

27 [Appending events - Kurrent Docs](#)

https://docs.kurrent.io/clients/node/legacy/v6.2/appending-events?utm_source=chatgpt.com

28 <https://kafka.apache.org/33/streams/core-concepts/>

<https://kafka.apache.org/33/streams/core-concepts/>

30 31 111 <https://www.kurrent.io/blog/how-to-get-the-current-entity-state-from-events>

<https://www.kurrent.io/blog/how-to-get-the-current-entity-state-from-events>

34 38 92 99 144 148 <https://redis.io/docs/latest/commands/xack/>

<https://redis.io/docs/latest/commands/xack/>

35 101 115 <https://redis.io/docs/latest/develop/using-commands/transactions/>

<https://redis.io/docs/latest/develop/using-commands/transactions/>

36 <https://redis.io/docs/latest/develop/programmability/>

<https://redis.io/docs/latest/develop/programmability/>

37 88 89 98 <https://redis.io/docs/latest/commands/set/>

<https://redis.io/docs/latest/commands/set/>

39 76 85 90 106 <https://redis.io/docs/latest/operate/rc/databases/configuration/clustering/>

<https://redis.io/docs/latest/operate/rc/databases/configuration/clustering/>

42 <https://ethereum.github.io/yellowpaper/paper.pdf>

<https://ethereum.github.io/yellowpaper/paper.pdf>

44 79 <https://docs.solana.com/implemented-proposals/readonly-accounts>

<https://docs.solana.com/implemented-proposals/readonly-accounts>

45 <https://github.com/clockworklabs/SpacetimeDB>

<https://github.com/clockworklabs/SpacetimeDB>

46 48 49 52 <https://spacetimedb.com/docs/functions/reducers/>

<https://spacetimedb.com/docs/functions/reducers/>

47 51 <https://spacetimedb.com/docs/subscriptions>

<https://spacetimedb.com/docs/subscriptions>

50 75 94 104 <https://github.com/clockworklabs/SpacetimeDB/releases>

<https://github.com/clockworklabs/SpacetimeDB/releases>

53 54 <https://docs.ipfs.tech/concepts/lifecycle/>

<https://docs.ipfs.tech/concepts/lifecycle/>

55 59 <https://www.csa.iisc.ac.in/~raghavan/CleanedPods2021/crdt-shapiro-2011.pdf>

<https://www.csa.iisc.ac.in/~raghavan/CleanedPods2021/crdt-shapiro-2011.pdf>

56 <https://github.com/automerger/automerger>

<https://github.com/automerger/automerger>

57 <https://docs.ipfs.tech/concepts/merkle-dag/>

<https://docs.ipfs.tech/concepts/merkle-dag/>

58 <https://automerge.org/automerge-binary-format-spec/>

<https://automerge.org/automerge-binary-format-spec/>

60 113 125 <https://arxiv.org/pdf/1702.08734>

<https://arxiv.org/pdf/1702.08734>

61 <https://arxiv.org/pdf/1603.09320>

<https://arxiv.org/pdf/1603.09320>

62 95 112 <https://redis.io/docs/latest/develop/clients/nodejs/vecsearch/>

<https://redis.io/docs/latest/develop/clients/nodejs/vecsearch/>

63 97 <https://milvus.io/docs/ivf-pq.md>

<https://milvus.io/docs/ivf-pq.md>

64 67 <https://github.com/spotify/annoy>

<https://github.com/spotify/annoy>

65 96 128 <https://docs.weaviate.io/weaviate/config-refs/indexing/vector-index>

<https://docs.weaviate.io/weaviate/config-refs/indexing/vector-index>

66 131 132 <https://www.pinecone.io/blog/serverless-architecture/>

<https://www.pinecone.io/blog/serverless-architecture/>

70 84 93 107 142 4.4.3. Neighbor lists

https://docs.lammps.org/Developer_par_neigh.html?utm_source=chatgpt.com

71 77 108 124 <https://redis.io/docs/latest/develop/ai/search-and-query/vectors/>

<https://redis.io/docs/latest/develop/ai/search-and-query/vectors/>

72 81 151 https://docs.lammps.org/Developer_comm_ops.html

https://docs.lammps.org/Developer_comm_ops.html

74 141 https://box2d.org/doc_version_2_4/

https://box2d.org/doc_version_2_4/

80 103 109 <https://redis.io/docs/latest/develop/programmability/lua-api/>

<https://redis.io/docs/latest/develop/programmability/lua-api/>

86 <https://redis.io/docs/latest/develop/using-commands/keyspace/>

<https://redis.io/docs/latest/develop/using-commands/keyspace/>

87 91 <https://redis.io/docs/latest/develop/using-commands/multi-key-operations/>

<https://redis.io/docs/latest/develop/using-commands/multi-key-operations/>

100 <https://redis.io/active-active/>

<https://redis.io/active-active/>

102 https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/

https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/

117 <https://redis.io/blog/simplifying-streams-and-strings/>

<https://redis.io/blog/simplifying-streams-and-strings/>

118 <https://redis.io/docs/latest/develop/data-types/json/>

<https://redis.io/docs/latest/develop/data-types/json/>

- 119 <https://redis.io/docs/latest/develop/ai/search-and-query/indexing/>
<https://redis.io/docs/latest/develop/ai/search-and-query/indexing/>
- 120 <https://kafka.apache.org/42/design/design/>
<https://kafka.apache.org/42/design/design/>
- 121 <https://docs.nats.io/nats-concepts/core-nats>
<https://docs.nats.io/nats-concepts/core-nats>
- 122 [JetStream Model Deep Dive - NATS Docs](#)
https://docs.nats.io/using-nats/developer/develop_jetstream/model_deep_dive?utm_source=chatgpt.com
- 123 [JetStream - NATS Docs](#)
https://docs.nats.io/nats-concepts/jetstream?utm_source=chatgpt.com
- 126 <https://milvus.io/docs/hnsw.md>
<https://milvus.io/docs/hnsw.md>
- 127 <https://docs.weaviate.io/weaviate/concepts/vector-index>
<https://docs.weaviate.io/weaviate/concepts/vector-index>
- 129 130 <https://qdrant.tech/documentation/concepts/indexing/>
<https://qdrant.tech/documentation/concepts/indexing/>
- 134 <https://doc.akka.io/libraries/akka-core/current/persistence.html>
<https://doc.akka.io/libraries/akka-core/current/persistence.html>
- 135 <https://docs.rs/actix>
<https://docs.rs/actix>
- 136 <https://github.com/cloudstateio/cloudstate>
<https://github.com/cloudstateio/cloudstate>
- 138 <https://hoomd-blue.readthedocs.io/en/v2.9.7/mpi.html>
<https://hoomd-blue.readthedocs.io/en/v2.9.7/mpi.html>
- 139 <https://ccl.northwestern.edu/netlogo/docs/faq.html>
<https://ccl.northwestern.edu/netlogo/docs/faq.html>
- 140 149 <https://www.canadabay.nsw.gov.au/sites/default/files/2018-11/simulation.pdf>
<https://www.canadabay.nsw.gov.au/sites/default/files/2018-11/simulation.pdf>